13
Meccanismi Avanzati
Classi innestate e enumerazioni

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Illustrare meccanismi avanzati della programmazione OO
- Dare linee guida sul loro utilizzo

Argomenti

- Enumerazioni
- Classi innestate statiche
- Inner class
- Classi locali
- Classi anonime
- Mappe del Collection Framework

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

Classi innestate statiche – idea e terminologia

Principali elementi

- Dentro una classe `A`, chiamata outer è possibile innestare la definizione di un'altra classe `B`, chiamata innestata (statica) – in inglese, static nested
- `B` viene quindi vista come se fosse una proprietà `statica` di `A` (richiamabile via `A`, come: tipo, per le `new` e le chiamate statiche)

```
// situazione di partenza
class A {...}
class B {...}
```

```
// modifica, usando le inner class
class A {
    ...
    static class B { .. }
}
```

Classi innestate statiche – casistica

Possibilità di innestamento

- Anche una interfaccia può fungere da `Outer`
- Si possono innestare anche interfacce
- Il nesting può essere multiplo e/o multilivello
- L'accesso alle classi/interfacce innestate statiche avviene con sintassi `Outer.A`, `Outer.B`, `Outer.I`, `Outer.A.C`

```
class Outer {
    ...
    static class A { .. static class C{..} ..}
    static class B {..}
    interface I {..} // static è implicito
}
```

Classi innestate statiche – accesso

Uso

- L'accesso alle classi/interfacce innestate statiche avviene con sintassi `Outer.StaticNested`
- Da dentro `Outer` si può accedere anche direttamente con `StaticNested`
- L'accesso da fuori `Outer` di `StaticNested` segue le regole del suo modificatore d'accesso
- Esterna e interna si vedono a vicenda anche le proprietà `private`

```
class Outer {
    ...
    static class StaticNested {
        ...
    }
}
..
Outer.StaticNested obj = new Outer.StaticNested(...);
```

Motivazioni

Una necessità generale

Vi sono situazioni in cui per risolvere un singolo problema è opportuno generare più classi, e non si vuole affiancarle solo come classi dello stesso package

Almeno tre motivazioni (non necessariamente contemporanee)

- Evitare il proliferare di classi in un package, specialmente quando solo una di queste debba essere pubblica
- Migliorare l'incapsulamento, con un meccanismo per consentire un accesso locale anche a proprietà `private`
- Migliorare la leggibilità, inserendo classi là dove serve (con nomi qualificati, quindi più espressivi)
- ..meglio comunque non abusare di questo meccanismo

Caso 1

Specializzazioni come classi innestate

- La classe astratta, o comunque base, è la outer
- Alcune specializzazioni ritenute frequenti e ovvie vengono innestate, ma comunque rese pubbliche
- due implicazioni:
  - schema di nome delle inner class
  - possibilità di accedere alle proprietà statiche

Esempio

- `Counter`, `Counter.Bidirectional`, `Counter.Multi`

Note

Un sintomo della possibilità di usare le classi nested per questo caso è quando ci si trova a costruire classi diverse costuite da un nome composto con una parte comune (`Counter`, `BiCounter`, `MultiCounter`)

Classe Counter e specializzazioni innestate (1/2)

```
public class Counter {

  private int value; // o protected..

  public Counter(int initialValue) {
    this.value = initialValue;
  }

  public void increment() {
    this.value++;
  }

  public int getValue() {
    return this.value;
  }

  public static class Multi extends Counter{
      ... // solito codice
  }

  public static class Bidirectional extends Counter{
      ... // solito codice
  }
}
```

Classe Counter e specializzazioni innestate (2/2)

```
public class Counter {

  ...
  // Codice della classe senza modifiche..
  public static class Multi extends Counter{

      public Multi(int initialValue){
        super(initialValue);
      }

      public void multiIncrement(int n){
        for (int i=0;i<n;i++){
            this.increment();
        }
      }
  }
    ...
  public static class Bidirectional extends Counter{
      ... // solito codice
  }
}
```

Uso di Counter e specializzazioni innestate

```
public class UseCounter {

  public static void main(String[] args) {
    final List<Counter> list = new ArrayList<>();
    list.add(new Counter(100));
    list.add(new Counter.Bidirectional(100));
    list.add(new Counter.Multi(100));

    for (final Counter c : list){
      c.increment();
    }

  }

}
```

Caso 2

Necessità di una classe separata ai fini di ereditaerità

In una classe potrebbero servire sotto-comportamenti che debbano:

- implementare una data interfaccia
- estendere una data classe

Esempio

- `Range`, `Range.Iterator`

Nota

In tal caso spesso tale classe separata non deve essere visibile dall'esterno, quindi viene indicata come `private`

Classe Range e suo iteratore (1/2)

```
public class Range implements Iterable<Integer>{

    final private int start;
    final private int stop;

    public Range(final int start, final int stop){
      this.start = start;
      this.stop = stop;
    }

    public java.util.Iterator<Integer> iterator(){
      return new Iterator(this.start,this.stop);
    }

    private static class Iterator
              implements java.util.Iterator<Integer>{
        ...
    }
}
```

Classe Range e suo iteratore (2/2)

```
public class Range implements Iterable<Integer>{
    ...
    private static class Iterator
              implements java.util.Iterator<Integer>{

        private int current;
        private final int stop;

        public Iterator(final int start, final int stop){
          this.current = start;
          this.stop = stop;
        }

        public Integer next(){
          return this.current++;
        }

        public boolean hasNext(){
          return this.current <= this.stop;
        }

        public void remove(){}
    }
}
```

Uso di Range

```
1 public class UseRange{
2     public static void main(String[] s){
3       for (final int i: new Range(5,12)){
4           System.out.println(i);
5           // 5 6 7 8 9 10 11 12
6       }
7     }
8 }
```

Caso 3

Necessità di comporre una o più classi diverse

- Ognuna realizzi un sotto-comportamento
- Per suddividere lo stato dell'oggetto
- Tali classi non utilizzabili indipendentemente dalla outer

Esempio tratto dal Collection Framework

- `Map`, `Map.Entry`
- (una mappa è "osservabile" come set di entry)

Riassunto classi innestate statiche

Principali aspetti

- Da fuori (se pubblica) vi si accede con nome `Outer.StaticNested`
- `Outer` e `StaticNested` sono co-locate: si vedono le proprietà `private`

Motivazione generale

- Voglio evitare la proliferazione di classi nel package
- Voglio sfruttare l'incapsulamento

Motivazione per il caso `public`

- Voglio enfatizzare i nomi `Out.C1`, `Out.C2`,..

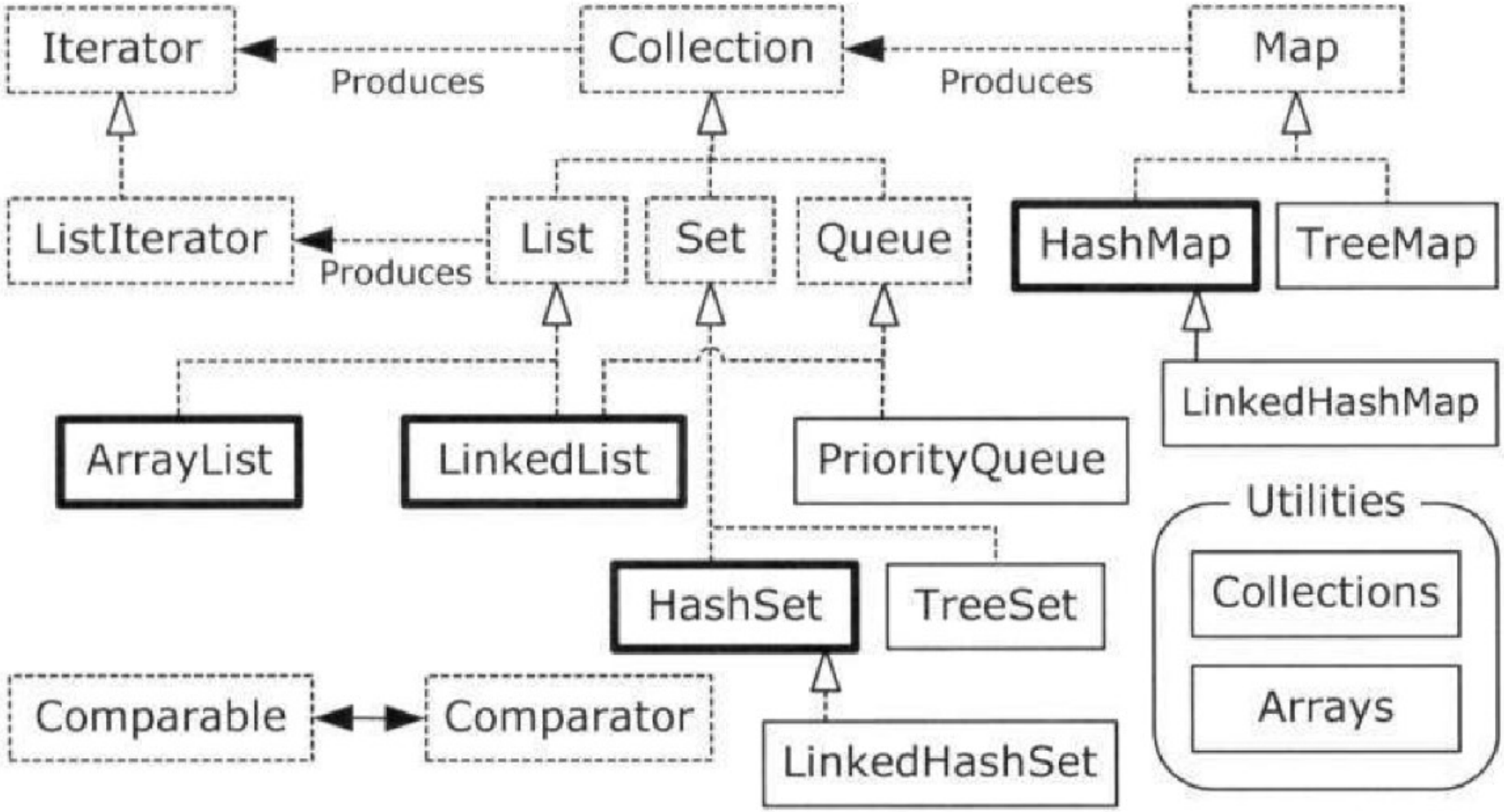
Motivazione per il caso `private` – è il caso più frequente

- Voglio realizzare una classe a solo uso della outer, invisibile alle altre classi del package

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

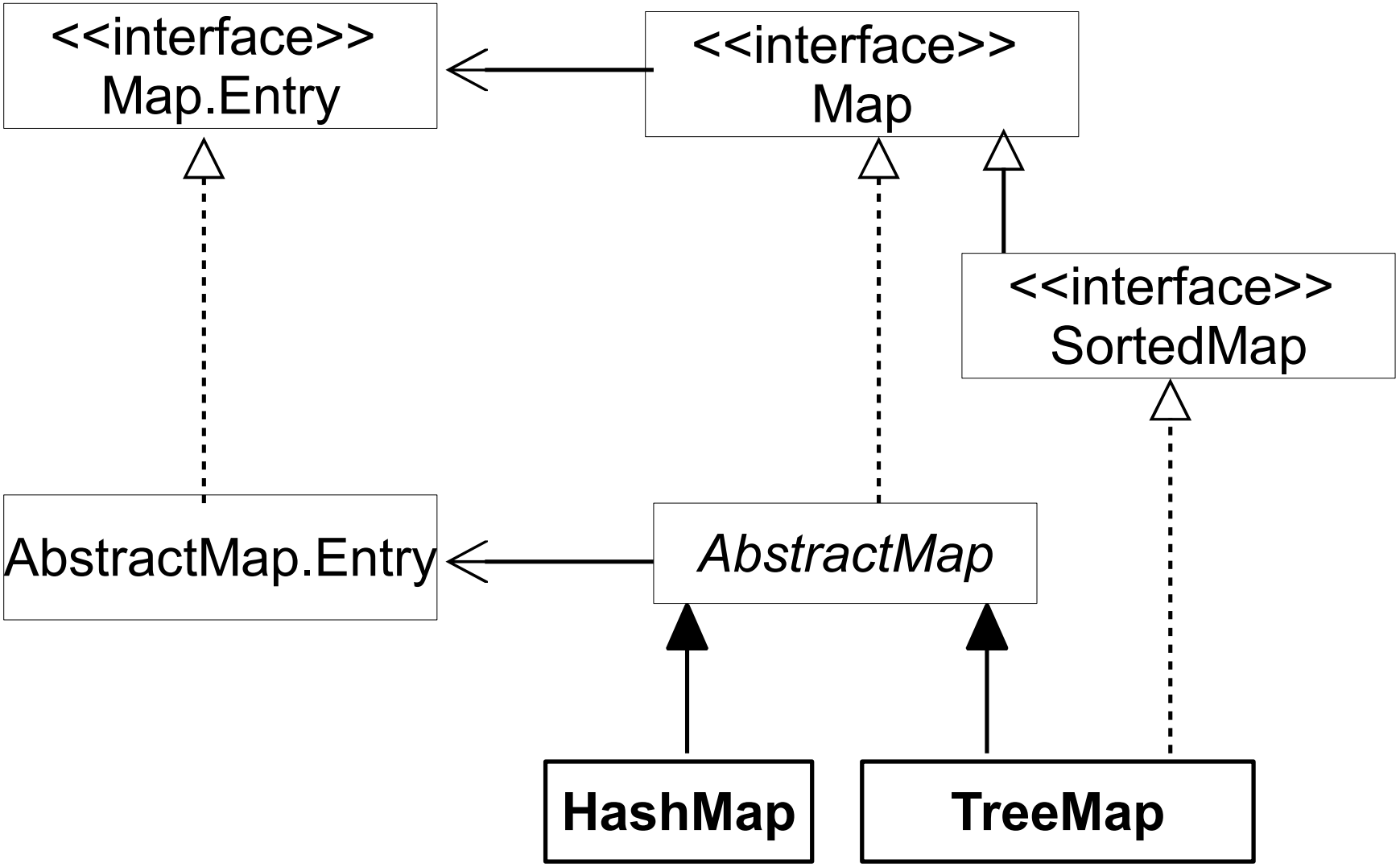
JCF – struttura semplificata



Map

```
public interface Map<K,V> {

    // Query Operations
    int size();
    boolean isEmpty();
    boolean containsKey(Object key);          // usa Object.equals
    boolean containsValue(Object value);      // usa Object.equals
    V get(Object key);                        // accesso a valore

    // Modification Operations
    V put(K key, V value);            // inserimento chiave-valore
    V remove(Object key);             // rimozione chiave(-valore)

    // Bulk Operations
    void putAll(Map<? extends K, ? extends V> m);
    void clear();                     // cancella tutti

    // Views
    Set<K> keySet();                      // set di chiavi
    Collection<V> values();               // collezione di valori
    Set<Map.Entry<K, V>> entrySet();      // set di chiavi-valore

    interface Entry<K,V> {...}            // public static implicito!
}
```

Implementazione mappe – UML



Map.Entry

Ruolo di Map.Entry

- Una mappa può essere vista come una collezione di coppie chiave-valore, ognuna incapsulata in un Map.Entry
- Quindi, una mappa è composta da un set di Map.Entry

```
public interface Map<K,V> {

    ...

    Set<Map.Entry<K, V>> entrySet();

    interface Entry<K,V> { // public e static implicite!

        K getKey();
        V getValue();
        V setValue(V value);

    }
}
```

Uso di Map.Entry

```java
public class UseMap2 {

  public static void main(String[] args) {

    // Al solito, uso una incarnazione, ma poi lavoro sull'interfaccia
    final Map<Integer, String> map = new HashMap<>();
    // Una mappa è una funzione discreta
    map.put(345211, "Bianchi");
    map.put(345122, "Rossi");
    map.put(243001, "Verdi");

    for (final Map.Entry<Integer, String> entry : map.entrySet()) {
      System.out.println(entry.getClass());
      System.out.println(entry.getKey());
      System.out.println(entry.getValue());
      entry.setValue(entry.getValue()+"...");
    }
    System.out.println(map);
    // {345211=null, 243001=null, 345122=null}
  }
}
```

La classe `AbstractMap`

In modo simile a `AbstractSet`

- Fornisce una implementazione scheletro per una mappa
- Necessita di un solo metodo da implementare: `entrySet()`
- Così facendo si ottiene una mappa iterabile e non modificabile
- Per fare modifiche è necessario ridefinire altri metodi..

Una semplice specializzazione di AbstractMap

```java
public class CapitalsMap extends AbstractMap<String,String>{

  private static final Set<Map.Entry<String,String>> set;

  // inizializzatore statico..
  // usato per inizializzare i campi statici
  static{
    // costruisce il valore di set
    set = new HashSet<>();
    set.add(new AbstractMap.SimpleEntry<>("Italy","Rome"));
    set.add(new AbstractMap.SimpleEntry<>("France","Paris"));
    set.add(new AbstractMap.SimpleEntry<>("Germany","Berlin"));
  }

  public CapitalsMap(){}

  // Questo è l'unico metodo che è necessario implementare
  public Set<java.util.Map.Entry<String, String>> entrySet() {
    return set;
  }

}
```

UseCapitalsMap

```
1  public class UseCapitalsMap{
2
3    public static void main(String[] args){
4      CapitalsMap cmap = new CapitalsMap();
5      System.out.println("Capital of Italy: "+cmap.get("Italy"));
6      System.out.println("Capital of Spain: "+cmap.get("Spain"));
7      System.out.println("All CapitalsMap: "+cmap);
8
9      // Iterazione "lenta" su una mappa
10     for (final String key: cmap.keySet()){
11       System.out.println("K,V: "+key+" "+cmap.get(key));
12     }
13
14     // Iterazione veloce su una mappa
15     for (final Map.Entry<String, String> entry: cmap.entrySet(){
16       System.out.println("E: "+entry+" "+entry.getKey()+" "+entry.getValue()
       );
17     }
18   }
19 }
```

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

Inner Class – idea

Principali elementi

- Dentro una classe `Outer`, è possibile innestare la definizione di un'altra classe `InnerClass`, senza indicazione `static`!
- `InnerClass` è vista come se fosse una proprietà non-statica di `Outer` al pari di altri campi o metodi
- L'effetto è che una istanza di `InnerClass` ha sempre un riferimento ad una istanza di `Outer` (enclosing instance) che ne rappresenta il contesto, accessibile con la sintassi `Outer.this`, e ai suoi campi (privati)

```
class Outer {
    ...
    class InnerClass { // Nota.. non è static!
        ...
        // ogni oggetto di InnerClass avrà un riferimento ad
        // un oggetto di Outer, denominato Outer.this
    }
}
```

Un semplice esempio

```java
public class Outer {

  private int i;

  public Outer(int i){
    this.i=i;
  }

  public Inner createInner(){
    return new Inner();
    // oppure: return this.new Inner();
  }

  public class Inner {

    private int j = 0;

    public void update(){
      // si usa l'oggetto di outer..
      this.j = this.j + Outer.this.i;
    }

    public int getValue(){
      return this.j;
    }
  }
}
```

Uso di Inner e Outer

```
public class UseOuter {

  public static void main(String[] args) {
    Outer o = new Outer(5);
    Outer.Inner in = o.new Inner();
    System.out.println(in.getValue()); // 0
    in.update();
    in.update();
    System.out.println(in.getValue()); // 5

    Outer.Inner in2 = new Outer(10).createInner();
    in2.update();
    in2.update();
    System.out.println(in2.getValue()); // 20
  }
}
```

Enclosing instance – istanza esterna

Gli oggetti delle inner class

- Sono creati con espressioni:
  `<obj-outer>.new <classe-inner>(<args>)`
- (la parte `<obj-outer>` è omettibile quando sarebbe `this`)
- Possono accedere all'enclosing instance con notazione
  `<classe-outer>.this`

Motivazioni: quelle relative alle classi innestate statiche, più..

- ...quando è necessario che ogni oggetto inner tenga un riferimento all'oggetto outer
- pragmaticamente: usato quasi esclusivamente il caso `private`

Esempio

- La classe `Range` già vista usa una static nested class, che però ben usufruirebbe del riferimento all'oggetto di `Range` che l'ha generata

Una variante di Range

```java
public class Range2 implements Iterable<Integer> {

  private final int start;
  private final int stop;

  public Range2(final int start, final int stop) {
    this.start = start;
    this.stop = stop;
  }

  public java.util.Iterator<Integer> iterator() {
    return this.new Iterator();
  }

  private class Iterator implements java.util.Iterator<Integer> {

    private int current;

    public Iterator() {
      this.current = Range2.this.start; // this.current = start
    }

    public Integer next() {
      return this.current++;
    }

    public boolean hasNext() {
      return this.current <= Range2.this.stop;
    }

    public void remove() {}
  }
```

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

Classi locali – idea

Principali elementi

- Dentro un metodo di una classe `Outer`, è possibile innestare la definizione di un'altra classe `LocalClass`, senza indicazione `static`!
- La `LocalClass` è a tutti gli effetti una inner class (e quindi ha enclosing instance)
- In più, la `LocalClass` "vede" anche le variabili nello scope del metodo in cui è definita, usabili solo se final, o se "di fatto finali"

```
class Outer {
    ...
    void m(final int x){
        final String s=..;
        class LocalClass { // Nota.. non è static!
        ... // può usare Outer.this, s e x
        }
        LocalClass c=new LocalClass(...);
    }
}
```

Range tramite classe locale

```
public class Range3 implements Iterable<Integer>{

    private final int start;
    private final int stop;

    public Range3(final int start, final int stop){
      this.start = start;
      this.stop = stop;
    }

    public java.util.Iterator<Integer> iterator(){
      class Iterator implements java.util.Iterator<Integer>{

            private int current;

            public Iterator(){
              this.current = Range3.this.start;
            }

            public Integer next(){
              return this.current++;
            }

            public boolean hasNext(){
              return this.current <= Range3.this.stop;
            }

            public void remove(){}
        }
      return new Iterator();
    }
}
```

Classi locali – motivazioni

Perché usare una classe locale invece di una inner class

- Tale classe è necessaria solo dentro ad un metodo, e lì la si vuole confinare
- È eventualmente utile accedere anche alle variabili del metodo

Pragmaticamente

- Mai viste usarle.. si usano invece le classi anonime..

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

Classi anonime – idea

Principali elementi

- Con una variante dell'istruzione `new`, è possibile innestare la definizione di un'altra classe senza indicarne il nome
- In tale definizione non possono comparire costruttori
- Viene creata al volo una classe locale, e da lì se ne crea un oggetto
- Tale oggetto, come per le classi locali, ha enclosing instance e "vede" anche le variabili final (o di fatto finali) nello scope del metodo in cui è definita

```
class C {
    ...
    Object m(final int x){
        return new Object(){
            public String toString(){ return "Valgo "+x; }
        }
    }
}
```

Range tramite classe anonima – la soluzione ottimale

```java
public class Range4 implements Iterable<Integer>{

    private final int start;
    private final int stop;

    public Range4(final int start, final int stop){
      this.start = start;
      this.stop = stop;
    }

    public java.util.Iterator<Integer> iterator(){
      return new java.util.Iterator<Integer>(){
            // Non ci può essere costruttore!
            private int current = start; // o anche Range4.this.start

            public Integer next(){
              return this.current++;
            }
            public boolean hasNext(){
              return this.current <= stop; // o anche Range4.this.stop
            }
            public void remove(){}
        }; // questo è il ; del return!!
    }
}
```

Classi anonime – motivazioni

Perchè usare una classe anonima?

- Se ne deve creare un solo oggetto, quindi è inutile anche solo nominarla
- Si vuole evitare la proliferazione di classi
- Tipicamente: per implementare "al volo" una interfaccia

Altro esempio: classe anonima da Comparable

```
1  public class UseSort {
2
3    public static void main(String[] args) {
4      final List<Integer> list = Arrays.asList
       (10,40,7,57,13,19,21,35);
5      System.out.println(list);
6      // classe anonima a partire da una interfaccia
7      Collections.sort(list,new Comparator<Integer>(){
8        public int compare(Integer a,Integer b){
9          return a-b;
10       }
11     });
12     System.out.println(list);
13
14     Collections.sort(list,new Comparator<Integer>(){
15       public int compare(Integer a,Integer b){
16         return b-a;
17       }
18     });
19     System.out.println(list);
20   }
21 }
```

Riassunto e linee guida

Inner class (e varianti)

Utili quando si vuole isolare un sotto-comportamento in una classe a sé, senza dichiararne una nuova che si affianchi alla lista di quelle fornite dal package, ma stia "dentro" una classe più importante

Se deve essere visibile alle altre classi

- Quasi sicuramente, una static nested class

Se deve essere invisibile da fuori

- Si sceglie uno dei quattro casi a seconda della visibilità che la inner class deve avere/dare
  - static nested class: solo parte statica
  - inner class: anche enclosing class, accessibile ovunque dall'outer
  - local class: anche argomenti/variabili, accessibile da un solo metodo
  - anonymous class: per creare un oggetto, senza un nuovo costruttore

Preview lambda expressions

Un pattern molto ricorrente

- Avere classi anonime usate per incapsulare metodi "funzionali"
- Java 8 introduce le lambda come notazione semplificata
- È il punto di partenza per la combinazione OO + funzionale

```
public class UseSortLambda {

  public static void main(String[] args) {
    final List<Integer> list = Arrays.asList
    (10,40,7,57,13,19,21,35);
    System.out.println(list);
    // classe anonima a partire da una interfaccia
    Collections.sort(list,(a,b)->a-b);
    System.out.println(list);

    Collections.sort(list,(a,b)->b-a);
    System.out.println(list);
  }
}
```

Outline

1. Classi innestate statiche
2. Il caso delle `java.util.Map`
3. Inner Class
4. Classi locali
5. Classi anonime
6. Enumerazioni

Enumerazioni

Motivazioni

- in alcune situazioni occorre definire dei tipi che possono assumere solo un numero fissato e limitato di possibili valori
- Esempi:
  - le cifre da 0 a 9, le regioni d'Italia, il sesso di un individuo, i 6 pezzi negli scacchi, i giorni della settimana, le tipologie di camere di un hotel, le scuole di un ateneo, eccetera

Possibili realizzazioni in Java

- usare delle `String` rappresentando il loro nome: quasi assurdo
- usare degli `int` per codificarli (come in C): poco leggibile
- usare delle classi astratte, e una concreta per valore: prolisso

Enumerazioni: `enum { ...}`

- consentono di elencare i valori, associando ad ognuno un nome
- è possibile collegare metodi e campi ad ogni "valore"

Esempio classe Persona: 1/2

no !

```java
public class Persona {
  private final String nome;
  private final String cognome;
  private final String regione;

  public Persona(String nome, String cognome, String regione) {
    this.nome = nome;
    this.cognome = cognome;
    this.regione = regione;
  }

  public String toString() {
    return "[" + nome + "," + cognome + "," + regione + "]";
  }
```

Esempio classe Persona: 2/2

```java

  public boolean isIsolano() { // Confronto molto lento!!
    return (this.regione.equals("Sardegna") ||
        this.regione.equals("Sicilia"));
  }

  public static List<Persona> fromRegione(
      final Collection<Persona> coll,
      final String regione
  ) { // nota questa possibile formattazione
    final List<Persona> list = new ArrayList<>();
    for (final Persona persona : coll) {
      if (persona.regione.equals(regione)) { // Confronto lento!!
        list.add(persona);
      }
    }
    return list;
  }
}
```

UsePersona

```
public class UsePersona {
  public static void main(String[] args){
    final ArrayList<Persona> list = new ArrayList<>();
    list.add(new Persona("Mario","Rossi","Emilia-Romagna"));
    list.add(new Persona("Gino","Bianchi","Sicilia"));
    list.add(new Persona("Carlo","Verdi","EmiliaRomagna"));
    // Errore sul nome non intercettabile
    final List<Persona> out = Persona.fromRegione(list,"Emilia-Romagna");
    System.out.println(list);
    // [[Mario,Rossi,Emilia-Romagna], [Gino,Bianchi,Sicilia],
    //    [Carlo,Verdi,EmiliaRomagna]]
    System.out.println(out);
    // [[Mario,Rossi,Emilia-Romagna]]
    for (final Persona p: list){
      if (p.isIsolano()){
        System.out.println(p);
      }
    }
    // [Gino,Bianchi,Sicilia]
  }
}
```

Soluzione alternativa, Persona: 1/2

No!

```java
import java.util.*;

public class Persona {

    public static final int LOMBARDIA = 0;
    public static final int EMILIA_ROMAGNA = 1;
    public static final int SICILIA = 2;
    public static final int SARDEGNA = 3;
    ...

    private final String nome;
    private final String cognome;
    private final int regione;

    public Persona(String nome, String cognome, int regione) {
      this.nome = nome;
      this.cognome = cognome;
      this.regione = regione;
    }

    private static String nomeRegione(int regione){
        switch (regione){
          case 0: return "Lombardia";
          case 1: return "Emilia-Romagna";
          ...
        }
    }
```

Soluzione alternativa Persona: 2/2

```
  public String toString() {
    return "[" + nome + "," + cognome + "," + nomeRegione(regione) + "]";
  }

  public boolean isIsolano(){
    // Confronto veloce!!
    return (this.regione == SARDEGNA ||
            this.regione == SICILIA);
  }

  public static List<Persona> fromRegione(Collection<Persona> coll,
                                          String regione){
    ArrayList<Persona> list = new ArrayList<>();
    for (Persona persona: coll){
        // Confronto veloce!!
      if (persona.regione == regione){
        list.add(persona);
      }
    }
    return list;
  }
}
```

Soluzione alternativa UsePersona

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class UsePersona {
5    public static void main(String[] args){
6      ArrayList<Persona> list = new ArrayList<>();
7      list.add(new Persona("Mario","Rossi",Persona.EMILIA_ROMAGNA));
8      list.add(new Persona("Gino","Bianchi",3));
9      list.add(new Persona("Carlo","Verdi",Persona.LOMBARDIA));
10     // Non si hanno errori sulle stringhe
11     // Non si può prevenire l'uso diretto di interi..
12     ...
13   }
14 }
```

Soluzione con polimorfismo e ereditarietà ♑ !

```
interface Regione {
   String getNome();
}

class EmiliaRomagna implements Regione {
   String getNome() {
      return "Emilia Romagna";
   }
}
class Lombardia implements Lombardia {
   String getNome() {
      return "Lombardia";
   }
}

class Persona {
   private String nome;
   private String cognome;
   private Regione regione;
   ...
}

class UsePersona {
   public static void main(String[] args){
      ArrayList<Persona> list = new ArrayList<>();
      list.add(new Persona("Mario","Rossi",new ElimiaRomagna());
      list.add(new Persona("Gino","Bianchi",new Lombardia());
      ...
```

Discussione

Approccio a stringhe

- Penalizza molto le performance spazio-tempo
- Può comportare errori gravi per scorrette digitazioni
- Difficile intercettare gli errori

Approccio a interi – soluzione pre-enumerazioni

- Buone performance ma cattiva leggibilità
- Può comportare comunque errori, anche se più difficilmente
- L'uso delle costante è un poco dispersivo

altri approcci: uso di classi diverse per ogni valore

- Impraticabile con un numero molto elevato di valori
- Comunque molto prolisso in termini di quantità di codice
- Tuttavia:
  - Previene gli errori che si possono commettere
  - Consente facilmente di aggiungere nuovi elementi

Soluzione via costanti e costruttore privato

```java
public class Regione {

  public static final Regione MARCHE = new Regione(0,"Marche");
  public static final Regione VENETO = new Regione(1,"Veneto");
  public static final Regione LOMBARDIA = new Regione(2,"Lombardia");

  public static final Regione[] VALUES =
      new Regione[] {MARCHE, VENETO, LOMBARDIA};

  private final int id;
  private final String name;

  private Regione(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public int getId() {
    return this.id;
  }

  public String getName() {
    return this.name;
  }

  public String toString() {
    return "Regione [id=" + id + ", name=" + name + "]";
  }
```

UseRegione

```java
import java.util.Arrays;

public class UseRegione {

  public static void main(String[] args) {
    // nella variabile regione, si possono usare solo 3 casi
    Regione regione = Regione.MARCHE;

    System.out.println(regione);
    // si ottengono gli array dei valori possibile
    System.out.println(Arrays.toString(Regione.VALUES));
    // è possibile accedere alla "prossima regione"
    System.out.println(Regione.VALUES[regione.getId()+1]);
  }

}
```

enum in Java

Un nuovo tipo di dato

- Simile ad una classe
- Realizza l'approccio a costanti e costruttore privato
- Ottime performance, l'oggetto è già disponibile
- Impedisce interamente errori di programmazione
- Il codice aggiuntivo da produrre non è elevato

Unica precauzione

- Andrebbero usate per insiemi di valori che difficilmente cambieranno in futuro
- Difficile modificare il codice successivamente

enum Regione

```java
public enum Regione {
  ABRUZZO, BASILICATA, CALABRIA, CAMPANIA, EMILIA_ROMAGNA,
  FRIULI_VENEZIA_GIULIA, LAZIO, LIGURIA, LOMBARDIA, MARCHE,
  MOLISE, PIEMONTE, PUGLIA, SARDEGNA, SICILIA, TOSCANA,
  TRENTINO_ALTO_ADIGE, UMBRIA, VALLE_D_AOSTA, VENETO;
}
```

```java
import java.util.*;

public class UseEnum {
  public static void main(String[] args) {
    final List<Regione> list = new ArrayList<>();

    list.add(Regione.LOMBARDIA);
    list.add(Regione.PIEMONTE);
    list.add(Regione.EMILIA_ROMAGNA);

    for (final Regione r: list){
      System.out.println(r.toString());
    }
  }
}
```

Persona con uso della enum

```java
public class Persona {
  private final String nome;
  private final String cognome;
  private final Regione regione;

  public Persona(String nome, String cognome, Regione regione) {
    this.nome = nome;
    this.cognome = cognome;
    this.regione = regione;
  }

  public boolean isIsolano(){
    // Confronto veloce!!
    return (this.regione == Regione.SARDEGNA
        || this.regione == Regione.SICILIA);
  }

  public static List<Persona> fromRegione(Collection<Persona> coll,
      Regione regione){
    final List<Persona> list = new ArrayList<>();
    for (final Persona persona: coll){
      if (persona.regione == regione){ // Nota l'== !!
        list.add(persona);
      }
    }
    return list;
  }

```

UsePersona con uso della enum

```
import java.util.*;

public class UsePersona {
  public static void main(String[] args){
    final ArrayList<Persona> list = new ArrayList<>();
    list.add(new Persona("Mario","Rossi",Regione.EMILIA_ROMAGNA));
    list.add(new Persona("Gino","Bianchi",Regione.SICILIA));
    list.add(new Persona("Carlo","Verdi",Regione.LOMBARDIA));
    final List<Persona> out =
        Persona.fromRegione(list,Regione.EMILIA_ROMAGNA);
    System.out.println(list);
    // [[Mario,Rossi,EMILIA_ROMAGNA], [Gino,Bianchi,SICILIA],
    // [Carlo,Verdi,LOMBARDIA]]
    System.out.println(out);
    // [[Mario,Rossi,EMILIA_ROMAGNA]]
    for (final Persona p: list){
      if (p.isIsolano()){
        System.out.println(p);
      }
    }
    // [Gino,Bianchi,SICILIA]
  }
}
```

UsePersona con import static

```java
import static it.unibo.apice.oop.p13advanced.enums.en2.Regione.*;

import java.util.*;

public class UsePersona2 {
  public static void main(String[] args){
    final ArrayList<Persona> list = new ArrayList<>();
    list.add(new Persona("Mario","Rossi",EMILIA_ROMAGNA));
    list.add(new Persona("Gino","Bianchi",SICILIA));
    list.add(new Persona("Carlo","Verdi",LOMBARDIA));
    final List<Persona> out = Persona.fromRegione(list,EMILIA_ROMAGNA);
    System.out.println(list);
    // [[Mario,Rossi,EMILIA_ROMAGNA], [Gino,Bianchi,SICILIA],
    // [Carlo,Verdi,LOMBARDIA]]
    System.out.println(out);
    // [[Mario,Rossi,EMILIA_ROMAGNA]]
    for (final Persona p: list){
      if (p.isIsolano()){
        System.out.println(p);
      }
    }
    // [Gino,Bianchi,SICILIA]
  }
}
```

Metodi di default per ogni enum

```java
import static it.unibo.apice.oop.p13advanced.enums.en2.Regione.*;

import java.util.*;


public class UseRegione {
  public static void main(String[] args) {
    final ArrayList<Regione> list = new ArrayList<>();
    // 4 modi di ottenere una Regione
    list.add(Regione.LOMBARDIA);
    list.add(SARDEGNA);
    list.add(Regione.valueOf("SICILIA"));
    list.add(Regione.values()[10]);

    for (final Regione r: list){
      System.out.println("toString "+r); // LOMBARDIA,...,MOLISE
      System.out.println("ordinale "+r.ordinal()); // 8, 13, 14, 10
      System.out.println("nome "+r.name()); // LOMBARDIA,...,MOLISE
      System.out.println("---");
    }

    for (final Regione r: Regione.values()){
      System.out.print(r+" "); // Stampa tutte le regioni
    }

  }
}
```

enum negli switch

```java
import static it.unibo.apice.oop.p13advanced.enums.en2.Regione.*;

import java.util.*;

public class UseRegione2 {
  public static void main(String[] args) {
    final ArrayList<Regione> list = new ArrayList<>();
    // 4 modi di ottenere una Regione
    list.add(Regione.LOMBARDIA);
    list.add(SARDEGNA);
    list.add(Regione.valueOf("SICILIA"));
    list.add(Regione.values()[10]);

    // Le enum sono usabili negli switch
    for (final Regione r : list) {
      switch (r) {
      case LOMBARDIA:
        System.out.println("Lombardia");
        break;
      case EMILIA_ROMAGNA:
        System.out.println("Emilia Romagna");
        break;
      default:
        System.out.println("Altre..");
      }
    }
  }
}
```

Metodi aggiuntivi nelle enum: Zona

```
import java.util.*;

public enum Zona {
  NORD, CENTRO, SUD;

  public List<Regione> getRegioni(){
    final ArrayList<Regione> list=new ArrayList<>();
    for (final Regione r: Regione.values()){
      if (r.getZona()==this){
        list.add(r);
      }
    }
    return list;
  }
}
```

Metodi e campi aggiuntivi nelle enum: Regione (1/2)

```java
import static it.unibo.apice.oop.p13advanced.enums.en3.Zona.*;

public enum Regione {
  ABRUZZO(CENTRO,"Abruzzo"),
  BASILICATA(SUD,"Basilicata"),
  CALABRIA(SUD,"Calabria"),
  CAMPANIA(SUD,"Campania"),
  EMILIA_ROMAGNA(NORD, "Emilia Romagna"),
  FRIULI_VENEZIA_GIULIA(NORD, "Friuli Venezia Giula"),
  LAZIO(CENTRO, "Lazio"),
  LIGURIA(NORD, "Liguria"),
  LOMBARDIA(NORD, "Lombardia"),
  MARCHE(CENTRO, "Marche"),
  MOLISE(SUD, "Molise"),
  PIEMONTE(NORD, "Piemonte"),
  PUGLIA(SUD, "Puglia"),
  SARDEGNA(SUD, "Sardegna"),
  SICILIA(SUD, "Sicilia"),
  TOSCANA(CENTRO, "Toscana"),
  TRENTINO_ALTO_ADIGE(NORD, "Trentino Alto Adige"),
  UMBRIA(CENTRO, "Umbria"),
  VALLE_D_AOSTA(NORD,"Valle D'Aosta"),
  VENETO(NORD,"Veneto");

  private final Zona z;
  private final String actualName;

  private Regione(final Zona z, final String actualName){
    this.z = z;
    this.actualName = actualName;
  }
```

Metodi e campi aggiuntivi nelle enum: Regione (2/2)

```
  SICILIA(SUD, "Sicilia"),
  TOSCANA(CENTRO, "Toscana"),
  TRENTINO_ALTO_ADIGE(NORD, "Trentino Alto Adige"),
  UMBRIA(CENTRO, "Umbria"),
  VALLE_D_AOSTA(NORD,"Valle D'Aosta"),
  VENETO(NORD,"Veneto");

  private final Zona z;
  private final String actualName;

  private Regione(final Zona z, final String actualName){
    this.z = z;
    this.actualName = actualName;
  }

  public Zona getZona(){
    return this.z;
  }

  public String toString(){
    return this.actualName;
  }
}
```

Metodi e campi aggiuntivi nelle enum: UseZona

```
import static it.unibo.apice.oop.p13advanced.enums.en3.Zona.*;

public class UseZona {
  public static void main(String[] args) {
    for (Regione r: NORD.getRegioni()){
      System.out.println("toString "+r);
      // Emilia Romagna,...,Veneto
      System.out.println("nome "+r.name());
      // EMILIA_ROMAGNA,...,VENETO
      System.out.println("---");
    }
  }
}
```

Meccanismi per le `enum`

Riassunto

- Esistono metodi istanza e statici disponibili per `Enum`
- Si possono aggiungere metodi
- Si possono aggiungere campi e costruttori

Riguardando la `enum Regione`

- È una classe standard, con l'indicazioni di alcuni oggetti predefiniti
- I 20 oggetti corrispondenti alle regioni italiane

Quindi

- È possibile intuirne la realizzazione interna
- E quindi capire meglio quando e come usarli

⇒ In caso in cui i valori sono "molti e sono noti", oppure..

⇒ Anche se i valori sono pochi, ma senza aggiungere troppi altri metodi..

enum innestate

Motivazione

- Anche le `enum` (statiche) possono essere innestate in una classe o interfaccia o enum
- Questo è utile quando il loro uso è reputato essere confinato nel funzionamento della classe outer

Esempio

- `enum Regione` potrebbe essere inserita dentro `Persona`
- `enum Zona` potrebbe essere inserita dentro `Regione`